

CSE 231: Boa (Let-bindings and Scope)

Ranjit Jhala April 2, 2026

The Boa Language

Next: Boa which has

- **let bindings** with multiple variables
- **binary operators** (+, -, *)

This will let us learn about

- **Variable scoping** with lexical scope
- **Errors** for duplicate / unbound variables

Three Pieces of a Language

Recall! Three pieces define any language we compile:

1. **Concrete syntax** — the text the programmer writes
2. **Abstract syntax** — a Rust datatype our compiler uses
3. **Semantics** — what values programs produce

4. *Compile!*

Concrete Syntax

```

<expr> :=
| <number>
| (add1 <expr>)
| (sub1 <expr>)
| (let (<ident> <expr>) <expr>)
| <ident>

```

(let (x e₁) e₂)

string let x = e₁ { . e₂ }

-- def-var

-- use-var

s-expression parenthesized prefix notation.

- <number>s are base 10 and must be representable as an i32
- <ident>s are alphanumeric (+ hyphens/underscores); starting with a letter
- let has parens around the bind

QUIZ: Abstract Syntax

```

enum Expr {
    Number(i32),
    Add1(Box<Expr>),
    Sub1(Box<Expr>),
    // Fill in the cases for def-var, use-var
}

```

Recall Box<T> is a heap pointer — always a fixed 64-bit size, enabling recursive types.

QUIZ: Semantics*Program**Result*

```
(let (x 10)
  (add1 x))
```

→ 11

```
(let (x 10)
  (let (y (add1 x))
    (let (z (add1 y))
      (add1 z)))))
```

→ 13

```
(let (y (let (x 10)
  (add1 x)))
  (add1 y))
```

→ 12

```
(let (x 10)
  (let (x (add1 x))
    (add1 x)))
```

→ 12

```
(let (a 1)
  (let (c
    (let (b (add1 a))
      add1(b)))
    add1 c))
```

(let (x e₁) e₂)

→ 4

QUIZ: How many variables do we need to store ?*Program**Number of variables*

```
(let (x 10)
  (add1 x))
```

```
(let (x 10)
  (let (y (add1 x))
    (let (z (add1 y))
      (add1 z)))))
```

```
(let (y (let (x 10)
  (add1 x)))
  (add1 y))
```

```
(let (x 10)
  (let (x (add1 x))
    (add1 x)))
```

```
(let (a 1)
  (let (c
    (let (b (add1 a))
      add1(b)))
    add1 c))
```

Memory: Code, Globals, Heap and Stack

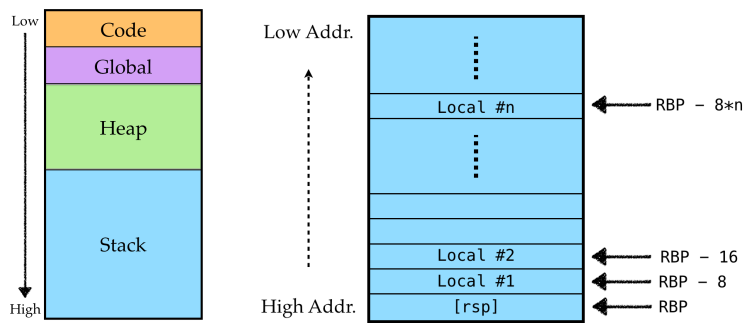


Figure 1: Store i^{th} stack-variable at address $\text{RBP} - 8 * i$.

Required

Map from vars ($x, y, z, \dots$) to stack positions (1, 2, 3 ...)

Solution

The structure of the `let` is stack-like too!

Maintain an Env that maps $\text{Id} \rightarrow \text{StackPosition}$

QUIZ

Which stack position do we store `c` in this program?

```
(let (a 1)
  (let (c
        (let (b (add1 a))
              add1(b)))
    add1 c))
```

Compilation Strategy: Definition

To compile **definition** `(let (x e1) e2)`

1. Compile `e1` using env (result in `rax`),
2. Move `rax` into `[RBP - 8 * i]`
3. Compile `e2` using env[`x := i`]

Compilation Strategy: Use

To compile **use** `x`

1. Look up `x` in env to get its stack position `i`
2. Load the value at `[RBP - 8 * i]` into `rax`

QUIZ: Compilation Examples

Program

```
(let (x 10)
  (add1 x))
```

Assembly

```
mov rax, 10
```

```
(let (x 10)
  (let (y (add1 x))
    (add1 y)))
```

```
mov rax, 10
```

```
(let (a 1)
  (let (c
        (let (b (add1 a))
          add1(b)))
    add1 c))
```

```
mov rax, 1
```